JalSetu: Heat Relief Dispatch

Judge briefing | 10 October 2026 | Working local prototype

Pitch in one sentence: JalSetu helps a relief coordinator turn confirmed water requests, limited stock and vehicle constraints into a checked delivery plan, compare alternatives, and revise remaining work after disruption while preserving reported deliveries.

The problem we address

A water-relief coordinator must decide who receives how much water, which vehicle takes it, and whether unloading can happen within the receiving window. A route alone cannot answer these questions. A spreadsheet can record requests, but a plausible allocation may still overload a vehicle, arrive too late or reuse water already delivered.

JalSetu combines allocation, one-trip routing, independent checking and operational records in one customer workflow. Its immediate customer is a coordinator of a small NGO, community, campus or shelter relief operation. This is a proposed user group; no partner organization or field adoption is claimed.

What exists today

Working locally	Prepared for AWS	Still unverified
Guided example; editable and saved projects; four strategies; approval; delivery reporting; recovery; revisions; readable exports.	Hosting and authenticated backend code, workflow, deployment templates, cost model and offline checks.	Resource creation, hosted sign-in/isolation, cloud workflow acceptance, genuine coordinator records and field benefit.

Our most useful measured result

Using a sourced public-road matrix with example relief inputs, JalSetu scheduled **12,000 modelled litres**, versus **11,100** for demand proportional. That is 900 additional scheduled litres, with **55.8 km** of estimated travel versus **44.0 km**. The travel tradeoff is part of the result, not omitted.

Status language for judges: "The application works locally and its AWS deployment is prepared and reviewed. It has not been deployed. Public-road estimates are real sourced data; operational requests and measured relief outcomes are not yet field-verified."

The project continued its existing codebase. It was not rebuilt. The current release keeps implementation bounded and explicit rather than adding autonomous dispatch, live vehicle tracking or an unvalidated AI decision maker.

A simple journey for the coordinator

Customer experience and the boundaries of this release

First use: try the example

The home page leads with **Try the example**. A short guide shows the next action and can be skipped, hidden or resumed. The customer works with names, litres, dates and clear choices. JSON editors, internal hashes and solver diagnostics are excluded from the customer interface.

Step	What the customer does	What JalSetu provides
1. Set up	Review Water & timing, Vehicles and Water requests.	A three-step setup with input checks and explicit operating rules.
2. Create a plan	Confirm the rules and calculate.	Four checked strategies, schedules, shortfalls and comparison metrics.
3. Decide	Compare tradeoffs, review and approve a plan.	A named approval and readable vehicle instructions. Approval does not dispatch a vehicle.
4. Record work	Report loading and cumulative delivered litres, with IST times.	An operational ledger that preserves accepted reports.
5. Respond	Record a failure, closure, reopening, recovery or return.	A checked depot-based revision for eligible remaining work.

What information is needed

The coordinator supplies depot water stock and opening hours, loading duration, vehicle capacities and availability, receiving-point coordinates and permissions, requested litres, minimum useful delivery, receiving capacity, priority with a reason, opening hours and unloading duration. Maps, population or weather do not establish a confirmed request.

Choices stay with the operator

The customer chooses whether unloading must **finish by closing** or only **start by closing**, and whether loading is **parallel** or **one at a time**. Evaluation uses explicit test settings; these settings are not a recommendation for a real organization.

> **Release envelope:** one depot, 1-5 vehicles, 1-20 receiving points, one operating day and one trip per vehicle, within the small Delhi-area input envelope. No second visits or active-position vehicle planning are claimed.

Saved projects can be archived and restored. Approved operations and reports remain separately retained. CSV imports have a readable preview before Apply, and exports are readable CSV sheets and printable HTML instructions.

How planning works

Constraint-based computation with independent checking

Implementation that is actually present

The frontend uses **React, TypeScript and Vite**. The local service is a loopback Node API. The shared planning core is **Python 3.12**, with Pydantic input validation and Google OR-Tools. Prepared infrastructure is AWS CDK v2 in JavaScript. The cloud API is a Python Lambda adapter; the local Node server is not the public deployment architecture.

Stage	Responsibility
Validate inputs	Reject malformed or unsupported records; exclude unconfirmed receiving points and unusable road connections.
Allocate water	Use integer planning units and per-vehicle packing. Keep allocations within stock, requests, useful-delivery minima and vehicle capacity.
Build schedules	Use OR-Tools routing with travel estimates, loading, service time, receiving windows and depot/vehicle limits.
Check independently	Recompute route feasibility and metrics rather than trusting the solver output. Reject inconsistent quantities, times or summaries.
Explain and compare	Show facts, unmet requests and checked alternatives in plain language.

What the JalSetu strategy prioritizes

The allocation model first maximizes critical minimum commitments, then high-priority minimum commitments, then total scheduled litres. Same-class minimum fulfilment is a lower-order objective. Stable tie-breaking helps reproducibility. This is an explicit policy: it is not a universal definition of fairness.

Allocation and routing are separate bounded stages with repair and checks. The route search is heuristic. A validated plan is feasible under its declared inputs; it is not a proof of globally optimal dispatch. A search timeout is not proof that no possible plan exists.

Why show four strategies?

Equal share, demand proportional, priority first and JalSetu use the same stored travel matrix and independent checker. This makes the allocation policy and tradeoffs visible instead of hiding the decision behind one suggested answer.

Resource questions are recalculated

The app independently solves alternatives for more stock, an extra vehicle, an earlier start, later closing or shorter service, alongside a fresh baseline. It keeps ties and losses. These analysis results do not silently change saved resources or become approved operations.

What happens after approval

Preserve physical reports; revise only eligible remaining work

Water accounting is central

Checked conservation rule: depot water + onboard water + reported delivered water + stranded water = initial stock + signed stock adjustments.

Approval selects a plan. Loading moves water from the depot to a vehicle. A delivery report records a cumulative delivered total for a receiving point. Repeating the same total does not create a second delivery. Accepted physical reports retain exact integer litres; for example, 713 L stays 713 L even when planning uses 100-L increments.

Event	Result in the application
Vehicle failure	Undelivered loaded water becomes stranded. It is not automatically available to a new route.
Recovery at depot	An explicit recovery report makes the specified stranded litres available again.
Vehicle return	Remaining onboard water goes back to the depot and the vehicle's one trip ends.
Point closes	The closure persists, earlier delivery reports remain, and unmet request stays visible.
Point reopens	An explicit reopening report restores availability. Review and approval are still needed to reschedule eligible work.
Stock adjustment	A signed adjustment changes depot inventory and the conservation total transparently.

Replanning has deliberate limits

The selected recovery workflow replans from the depot using unused eligible vehicles and available depot water. Earlier positive delivery reports remain fixed. An active vehicle must return or be reported failed before a depot-based revision can proceed. The app does not invent a vehicle location, assume recovered water or overwrite delivered quantities.

Persistence and duplicate protection

The local adapter uses atomic file replacement, conditional record versions and request receipts. Approved operations, events and archived projects survive tested process restarts. Stale updates are rejected instead of overwriting newer work. Planning jobs and unsaved changes have different persistence limits and can require recalculation after restart.

The local adapter is unauthenticated and intended for a trusted local machine. It is not a suitable public multi-user server or a claim of safe storage for confidential real-world records. Hosted ownership and access checks are prepared but still require live acceptance.

Real data: precisely what is real

Data provenance and the existing path to stronger proof

Data category	What we have	What it proves
Public road estimates	One frozen OSRM/OpenStreetMap response for nine example coordinates: 81 origin/destination pairs.	The planner can consume sourced road-network distances and car travel estimates.
Operational example inputs	Example stock, requests, vehicles, priorities, windows and receiving locations.	Controlled software comparisons, not actual water demand or facility permission.
Operator-entered records	Blank local projects, manual entry, CSV preview and declared source/date/review.	An input workflow exists. No genuine coordinator dataset has been obtained.
Delivery outcomes	Reporting logic and labelled synthetic regression records.	Accounting behavior is tested. No independently verified field deliveries or impact.

Public-road collection details

The frozen response was retrieved on **10 October 2026 at 11:35 IST**. All 81 pairs were present; the greatest road snap was **58.75 m**, below the preselected 250-m research limit. The underlying map date was not returned and is unknown. The coordinates remain examples. A car routing profile does not establish tanker clearance, receiving access or current congestion.

Attribution: OpenStreetMap contributors; routing by OSRM, hosted by FOSSGIS. This was a bounded personal, non-commercial research request. Production provider suitability, licensing obligations and service guarantees need review before operational use. No automatic refresh or paid routing service was used.

How a coordinator can bring real records

1. Create a blank local record-based project. 2. Enter anonymized depot and vehicle facts. 3. Import confirmed receiving points through CSV preview, then Apply. 4. Record the source, date and review acknowledgement. 5. Explicitly request matching public-road estimates after acknowledging coordinate sharing. 6. Calculate and compare under declared rules.

Compare an original dispatch plan

More > Compare original plan preserves original stop order, quantities and loading starts, reconstructs timing with the same roads, and compares a fresh JalSetu plan on the same resources. Invalid original plans keep their issues visible and do not receive misleading feasible-plan deltas. Optional delivery records remain separate; missing outcomes stay unknown.

For credible field proof, obtain confirmed requests, inventory, vehicle facts, receiving permissions/windows, the original dispatch plan and separate timestamped outcomes. A paired retrospective comparison measures estimated planning differences; a supervised pilot is needed to observe actual operational benefit.

Measured results and their limits

Keep the favourable result, the travel cost, the ties and the losses

North Delhi example using sourced public-road estimates

Strategy	Scheduled L	Unmet L	Travel km	Lowest fulfilment
Equal share	10,800	5,300	49.1	0.0%
Demand proportional	11,100	5,000	44.0	0.0%
Priority first	10,800	5,300	39.3	0.0%
JalSetu	12,000	4,100	55.8	29.2%

Against demand proportional, JalSetu schedules **900 more modelled litres (+8.1%)**, reduces modelled unmet request by **900 L (18%)**, and adds **11.8 km** of estimated travel using the displayed rounded distances. These are one scenario's computed results, not delivered-water measurements or a general performance guarantee.

What the experiment covers

Five scenarios x four strategies x four travel settings produced **80 retained calculations**. Settings were the synthetic reference, the frozen public-road estimate and invented 1.5x/2x duration multipliers. The multipliers test sensitivity; they are not observed traffic. All 80 recorded results were feasible and passed independent route checks. The report renderer also rechecks input integrity and metrics.

Test rules: finish-by-close, parallel loading, 100-L increments, seed 42 and a five-second solver allowance. A separate synthetic evaluation records 20 strategy calculations. These collections overlap in purpose and are not 100 independent field trials.

Cases where the headline does not hold

With shorter receiving hours and 2x travel, all four strategies tie at **7,800 L**. In the stock-shortage/1.5x case all schedule **7,000 L**, but equal share has **30%** lowest fulfilment and JalSetu **0%**. Prioritizing minimum commitments and total litres can leave a point unserved.

Software verification

The latest full local checkpoint records **237 passing Python tests**, TypeScript/Ruff checks, frontend build, customer checks and real-process release/restart checks. Docker checks exercise the planner and Lambda runtime offline. Some cloud tests use stubs; none establish live AWS acceptance. Browser journeys were checked by development, not an external usability study.

A bounded local burst completed 100 health reads. Ten competing plan requests admitted one and returned busy for nine; that accepted calculation completed. This verifies bounded admission behavior, not production scale or hosted latency.

> Safe judge claim: "We have reproducible software evidence using sourced road estimates and labelled example relief inputs. Actual operational benefit and real-user usability still need a partner dataset and pilot."

The prepared AWS architecture

AWS hosts the application and backend; it does not decide the water allocation

Deployment status: all services below are prepared in code and templates, not a running JalSetu deployment. The latest instruction is "Keep prepared; do not deploy."

Path	Prepared components
Customer access	Browser -> Amplify-hosted React app -> Cognito sign-in
Authenticated API	Cognito token -> API Gateway -> API Lambda
Planning job	API Lambda -> Step Functions Standard -> worker Lambda
Private records	API/worker -> DynamoDB for state, ownership and receipts; S3 for snapshots/results
Failure visibility	CloudWatch logs + EventBridge terminal-failure repair
Repeatable deployment	CDK/CloudFormation + IAM roles + ECR container assets

The seven planning stages

Order	Recorded stage	What happens
1	LoadAndValidate	Load the owner/job snapshot and validate inputs.
2	GetRouteMatrix	Read the stored example travel matrix. No AWS routing call.
3	SolveStrategies	Calculate four bounded strategies in the shared Python core.
4	ValidatePlans	Independently check schedules and computed metrics.
5	PersistResult	Write results and conditional completion records.
6	ExplainResult	Produce deterministic, grounded facts. No Bedrock call.
7	CompleteJob	Make the final recorded job state available to the customer.

Why these services fit this product

A browser request should not have to keep a long planning calculation alive. The prepared API admits a bounded job, and a Standard workflow records progress and handles stage retries. The worker image packages Python and OR-Tools. Stored snapshots allow results to be traced back to their inputs, while record ownership and conditional writes protect shared application state.

Stable job identifiers and conditional stage receipts guard against lost responses and retries. A terminal-failure rule and expiry repair path address jobs that otherwise appear stuck. Event delivery is best effort, so recovery must be tested and cannot be described as an outage guarantee.

This release uses AWS for hosting, authentication, execution and storage. **Amazon Location and Amazon Bedrock are excluded.** The initial hosted demo remains example-only; local operator-record acquisition and historical comparison must not be advertised as accepted hosted features.

AWS services and access controls

Concrete roles in the prepared stack

Service	JalSetu role	Why it matters to the customer
AWS Amplify	Manual hosting of the built React bundle.	A web entry point without a permanently running frontend server.
Amazon Cognito	Admin-created users, authorization code with PKCE.	Sign-in before operational API access. No public self-registration.
API Gateway	Cognito-authorized REST endpoints; limited request rate.	A controlled boundary between the browser and backend.
AWS Lambda	API adapter and separate Python/OR-Tools worker image.	Runs requests and planning without administering a VM.
Step Functions	Standard seven-stage planning workflow.	Recorded progress and explicit retry/failure handling.
DynamoDB	Owner-scoped state, versions, job/operation receipts and expiry metadata.	Persisted records with stale-write and duplicate protection.
Amazon S3	Encrypted private snapshots and results; lifecycle policy.	Retained inputs/results without a public operational-data bucket.
CloudWatch	API, worker and gateway logs; one-week retention.	Investigate failures with correlation records.
EventBridge	Terminal workflow failure event -> repair handler.	A recovery path for failed, aborted or timed-out jobs.
ECR	Container image assets required for Lambda deployment.	Reproducible Python and solver dependencies.
CDK / CloudFormation / IAM	Repeatable stacks and reviewed deployment/service permissions.	Reviewable infrastructure and access boundaries.

Controls already represented in code

API methods require Cognito authorization; CORS is limited to the prepared frontend origin. Backend access uses the authenticated owner, with conditional updates and private storage. The browser receives public endpoint configuration, not broad AWS credentials. S3 public access is blocked; workflow payloads carry identifiers rather than full operational inputs.

The reviewed bootstrap avoids the default administrator execution policy. Application actions use resource prefixes where supported. Some creation/management operations still require regional wildcard scope, and role-policy administration is sensitive. No universal project-only permission boundary or security certification is claimed.

Still to verify live: two-user isolation, private export ownership/expiry, actual IAM permissions, sign-in/callback behavior, workflow persistence and terminal repair. Offline checks and successful template validation are preparation evidence, not these acceptance results.

Budget, readiness and remaining gates

A bounded example demo, currently paused before resource creation

Latest account observation: 10 October 2026

Read-only checks recorded an **active AWS free plan with $100 remaining credits**, working temporary local CLI access and **400 unreserved Lambda concurrency slots in Mumbai (ap-south-1)**. The separate request for 1,000 remains Case Opened; approval is not claimed. The observed quota is sufficient for the prepared reservations. CloudShell still showed verification trouble, but the local CLI path worked. Credentials were logged out/cleared afterward.

Seven-day model	Before allowances/credits	Interpretation
Small usage	$11.58	$1.58 estimated service usage + $10 contingency.
Higher usage	$70.84	$60.84 estimated service usage + $10 contingency.

All 18 regional rates in the model were rechecked and unchanged. The estimates depend on declared traffic, duration, storage and transfer assumptions. They are not actual bills or hard spending caps. Other account usage shares the credits. Preserve the free plan and zero out-of-pocket requirement; stop if a service requires an account upgrade.

Prepared operating safeguards

Worker concurrency 1; API concurrency 2; API throttle 5 requests/second with burst 10; 100 admitted planning jobs per day; six-minute workflow timeout. Worker: 2 GB, 120-second function timeout. API: 512 MB, 25-second timeout. Demo expiry must be within seven days. These limit exposure but do not guarantee a dollar ceiling.

Work area	Current status	Next gate
Local product: Phases 1/2/6/7	Selected local features built and checked.	External usability and hosted acceptance where applicable.
AWS: Phases 3/4	Adapters, templates, bundle and offline checks prepared.	Deployment authorization, resource creation and live acceptance.
Data/evaluation: Phases 5/8	Public-road research and local evaluation/record comparison built.	Genuine coordinator records, measured outcomes and partner feedback.
Scope/release: Phases 0/10	Scope, local launcher, restart gates and evidence prepared.	Actual cloud release, monitoring/cleanup and final release freeze.
Submission: Phase 9	Draft material and private backup exist.	Public publication/license/submission deferred; video excluded.

Expiry stops application requests; it does not delete hosting or retained resources. Future deployment needs fresh checks and explicit approval. Cloud cleanup must cover exact retained resources and avoid unrelated/shared infrastructure.

Ready-to-speak pitch and live demonstration

Use the customer workflow; keep the status statement explicit

Approximately two-minute pitch

"Imagine coordinating water deliveries during a heat-relief operation. You have limited water, a few vehicles and receiving points that close at different times. A short route is not enough: the quantities must fit the vehicles, unloading must fit the window, and already delivered water cannot be allocated again.

JalSetu brings those decisions into one guided workflow. The coordinator starts with a simple example, reviews water and timing, vehicles and confirmed requests, and compares four checked plans. They approve a plan, print readable instructions and record loading and deliveries. If a vehicle fails or a receiving point closes, JalSetu preserves earlier reports and revises only the work that is still eligible.

The planning core uses constraint-based allocation and routing, followed by an independent checker. On a North Delhi example using sourced public-road estimates, JalSetu scheduled 12,000 modelled litres versus 11,100 for demand proportional. It also required more estimated travel: 55.8 kilometres versus 44.0. We expose that tradeoff, and retain scenarios where other strategies tie or do better.

The app works locally. Its AWS architecture is prepared, with Amplify hosting, Cognito sign-in, API Gateway, Lambda and Step Functions, plus private DynamoDB and S3 records and failure logging. It is not deployed yet. AWS supports reliable application execution; no Bedrock or Amazon Location calls are part of this release.

Our next proof is a paired comparison using anonymized coordinator records, followed by a supervised pilot. We are looking for confirmed requests, inventory, the original dispatch plan and separate delivery outcomes so we can test actual benefit rather than claim it from examples."

A short local demonstration

Sequence	Show	Say
1	Try the example and the setup guide.	"No technical file editing is required."
2	Review rules; select explicit demo settings; create a plan.	"These are declared test rules, not assumed field policy."
3	Compare strategies, unmet request and travel.	"More scheduled litres can require more travel."
4	Approve; report loading and one delivery.	"Approval and reported completion are separate."
5	Show a prepared failure/closure example or saved run.	"Previously reported water stays recorded."
6	Open the public-road research panel.	"The road estimates are sourced; the relief inputs are examples."

For the measured public-road result use **More > About the example > What we checked on public roads**. The normal guided example still uses its synthetic matrix. Do not imply that generating its plan calls a live routing service or that a demo report represents an actual delivery.

Answers to likely judge questions

Specific answers that the evidence supports

Is this just a route planner?

It includes quantity allocation, receiving-window and loading rules, strategy comparison, independent validation, approval and operational accounting. Failure recovery preserves delivered and stranded litres. Its value is connecting those decisions, within a deliberately small scope.

Does it use real data?

It uses a real frozen OSRM/OpenStreetMap road estimate in the research comparison. Requests, stock, vehicles, priorities, windows and coordinates in that experiment are examples. Real operator-entry and original-plan comparison workflows exist locally, but no genuine partner dataset has been obtained.

Is JalSetu deployed on AWS?

No. The deployment code, adapters, image, templates, pricing and permission review are prepared. No JalSetu resources or hosted test users were created. The latest instruction was to keep deployment paused. Live acceptance is still required.

Why AWS instead of a single server?

The prepared design separates sign-in, API requests, bounded planning work, persisted state and workflow recovery using managed services. That fits a small web demo with traceable jobs. It is more infrastructure than the local prototype; the tradeoff is managed capabilities and clearer execution records, not a proven cost advantage.

Where is the AI? Is a model deciding priorities?

The runtime decisions use OR-Tools constraint optimization and routing, not a language model. Priorities and reasons are declared by the coordinator; explanations are deterministic facts. Bedrock is excluded. Codex was used as a development assistant and should be disclosed separately from runtime capabilities.

Can you prove it is fair or optimal?

We can show objective priorities, comparison metrics and feasible-plan validation. We cannot claim a universal fairness guarantee or globally optimal routes. Equal share performs better on minimum fulfilment in a retained shortage case. The coordinator must choose a policy consistent with their operation.

Will it scale or handle live tanker routing?

The accepted local envelope is one depot, up to five vehicles and twenty points, one trip each. Live GPS, multiple trips, tanker clearance, production provider guarantees and large-fleet performance have not been validated. The initial hosted release is intentionally bounded and example-only.

What would demonstrate real impact?

First, replay a real original plan against JalSetu with identical resources, rules and roads; retain infeasibility, losses and missing outcomes. Then use a supervised pilot to compare observed water delivered, unmet requests, arrival-window success, distance/time and coordinator effort. No measured field improvement is claimed today.

Evidence index and next proof

Traceable project sources | Prepared briefing, not a completed cloud release

Project documents used for this report

Reference	What to inspect
README / Build status	Current application scope, local operation and latest checkpoint. Older entries are historical.
Specification / Decisions	Original plan versus later accepted choices; the specification is not a completion checklist.
Phase 2 completion	Earlier local customer-workflow evidence.
Public-road checkpoint / all results	Source provenance, experiment settings, full retained comparisons and limits.
Synthetic evaluation	Five fixtures and four strategies with independently checked metrics.
Record workflow / Historical comparison	Local record intake, original-plan replay and latest 237-test checkpoint.
Local release	Production launcher, real-process restart/failure gates and persistence checks.
AWS foundation / Deployment review	Prepared service design, account observations, reviewed changes, pause and cleanup requirements.
Cost model / Rate recheck	Workload assumptions and dated official rate verification.

Primary external references

AWS: Amplify manual deployments; Cognito user pools; API Gateway Cognito authorization; Lambda container images; Step Functions workflow types.

AWS: DynamoDB conditional writes; S3 Block Public Access; Step Functions/EventBridge events; CDK bootstrap; Free-plan rules.

OSRM API; FOSSGIS routing usage policy; OpenStreetMap attribution and licence.

The next practical ask

Ask one coordinator for a single anonymized day of confirmed requests, stock, vehicle facts, permissions/windows and the original planned stops. Obtain optional actual delivery records separately. Agree on priorities and success metrics before calculating. This is the shortest path to a stronger, honest evidence case.

This report introduces no deployment, account changes or new evaluation run. It summarizes recorded evidence as of 10 October 2026. Video, public publication/license and submission remain deferred. Source links in the editable report resolve within the project; PDF project references are file links for local review.